KLS

Vishwanathrao Deshpande Institute of TechnologyHaliyal–581329

DATA STRUCTURES LABORATORY

[BCSL305]

for

III Semester B.E.

Asprescribedby

VISVESVARAYA TECHNOLOGICAL UNIVERSITY,BELAGAVI–590014

(For the Academic Year 2025-2026)

Prepared by

Prof. Ekata Shanbhag

Department of Computer Science & Engineering(AIML)

Index

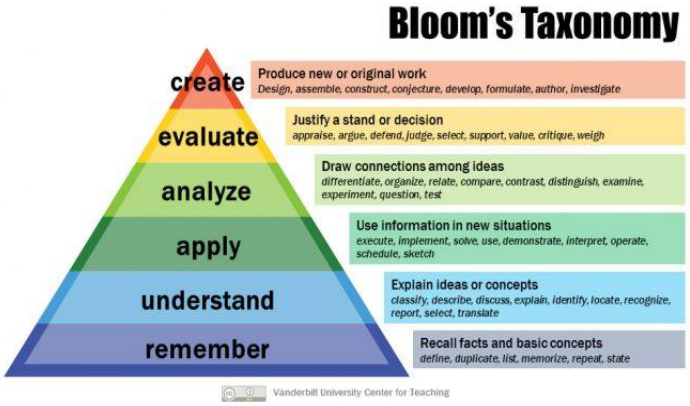
6	Develop a menu driven Program in C for the following operations on Circular QUEUE of Characters (Array Implementation of Queue with maximum size MAX) a. Insert an Element on to Circular QUEUE b. Delete an Element from Circular QUEUE c. Demonstrate Overflow and Underflow situations on Circular QUEUE d. Display the status of Circular QUEUE e. Exit Support the program with appropriate functions for each of the above operations.	23
7	Develop a menu driven Program in C for the following operations on Singly Linked List(SLL) of Student Data with the fields: USN, Name, Programme, Sem, PhNo a. Create a SLL of N Students Data by using front insertion. b. Display the status of SLL and count the number of nodes in it c. Perform Insertion / Deletion at End of SLL d. Perform Insertion / Deletion at Front of SLL(Demonstration of stack) e. Exit	25
8	Develop a menu driven Program in C for the following operations on Doubly Linked List(DLL) of Employee Data with the fields: SSN, Name, Dept, Designation, Sal, PhNo a. Create a DLL of N Employees Data by using end insertion. b. Display the status of DLL and count the number of nodes in it c. Perform Insertion and Deletion at End of DLL d. Perform Insertion and Deletion at Front of DLL e. Demonstrate how this DLL can be used as Double Ended Queue. f. Exit	28
9	Develop a Program in C for the following operations on Singly Circular Linked List (SCLL)with header nodes a. Represent and Evaluate a Polynomial P(x,y,z) = 6x2y2z-4yz5+3x3yz+2xy5z-2xyz3 b. Find the sum of two polynomials POLY1(x,y,z) and POLY2(x,y,z) and store the result in POLYSUM(x,y,z)Support the program with appropriate functions for each of the above operations.	29
10	Develop a menu driven Program in C for the following operations on Binary Search Tree(BST) of Integers . a. Create a BST of N Integers: 6, 9, 5, 2, 8, 15, 24, 14, 7, 8, 5, 2 b. Traverse the BST in In order, Preorder and Post Order c. Search the BST for a given element (KEY) and report the appropriate message d. Exit	32

11	Develop a Program in C for the following operations on Graph(G) of Cities a. Create a Graph of N cities using Adjacency Matrix. b. Print all the nodes reachable from a given starting node in a digraph using DFS/BFS method.	**32**
12	Given a File of N employee records with a set K of Keys (4-digit) which uniquely determine the records in file F. Assume that file F is maintained in memory by a Hash Table (HT) of memory locations with L as the set of memory addresses (2-digit) of locations in HT. Let thekeys in K and addresses in L are Integers. Develop a Program in C that uses Hash function H:K →L as H(K)=K mod m (remainder method), and implement hashingtechnique to map a given key K to the address space L. Resolve the collision (if any) usinglinear probing.	**32**
Virtual Lab Experiments		
1	Linked List	**36**
2	Binary Search Tree	**37**

PROGRAM OUTCOMES(POs)

Program Outcomes as defined by NBA (PO) Engineering Graduates will be able to:

1. Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
6. The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
7. Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8. Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9. Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
10. Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
11. Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
12. Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.



Vision (College)
To nurture talent and enrich society through excellence in technical education, research and innovation.
Mission (College)
1. To augment innovative Pedagogy, kindle quest for interdisciplinary learning & to enhance conceptual understanding. 2. To build competence, professional ethics & develop entrepreneurial thinking. 3. To strengthen Industry Institute Partnership & explore global collaborations. 4. To inculcate culture of socially responsible citizenship. 5. To focus on Holistic & Sustainable development.
Vision (Dept)
To lead the way in the creation and application of cutting-edge Computer science and engineering (AIML) technologies, advancing the frontiers of knowledge, and empowering future generations to drive innovation and transformation on a global scale.
Mission (Dept)
To train students with a strong conceptual understanding using innovative pedagogies, empowering them to excel in the dynamic fields of Artificial Intelligence and Machine Learning. To imbibe professional, research, and entrepreneurial skills with a commitment to the nation's development at large. To strengthen the industry-institute Interaction. To promote life–long learning with a sense of societal & ethical responsibilities.

Program Educational Objectives (PEO)	
PEO1	To develop an ability to identify and analyze the requirements of Computer Science and Engineering in design and providing novel engineering solutions.
PEO2	To develop abilities to work in team on multidisciplinary projects with effective communication skills, ethical qualities and leadership roles.
PEO3	To develop abilities for successful Computer Science Engineer and achieve higher career goals.

Program Specific Outcomes (PSO)	
PSO 1	To develop the ability to model real-world problems using appropriate data structure and suitable algorithms in the area of Data Processing, System Engineering, and Networking for varying complexity.
PSO 2	To develop an ability to use modern computer languages, environments and platforms in creating innovative career.

CO's And PO's Mapping Chart

Subject with code: DATA STRUCTURES LABORATORY (BCSL305) AY:2024-25

Semester: III

S.No.	Description	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2
1	Analyze various linear and non-linear data structures.	3											1	1	
2	Demonstrate the working nature of different types of data structures and their applications.	3	2	1	1								1	1	
3	Use appropriate searching and sorting algorithms for the give scenario.	3	2	1	1								1	1	
4	Apply the appropriate data structure for solving real world problems.	3	2	1	1								1	1	

Degree of compliance Low:1 Medium:2 High:3

Evaluation:

	Particulars	Marks	Total
CIA	Performance	20	**30**
	Journal	05	
	Viva-voce	05	
	Addon Course	05	**20**
	Virtual Lab Experiments	05	
	Lab IA	10	
	Grand Total		**50**

Mapping of Experiments with CO, PO and PSO

Sl. No.	Experiment Details	CO	PO	PSO
1	Develop a Program in C for the following:a) Declare a calendar as an array of 7 elements (A dynamically Created array) to represent 7 days of a week. Each Element of the array is a structure having three fields. The first field is the name of the Day (A dynamically allocated String), The second field is the date of the Day (A integer), the third field is the description of the activity for a particular day (A dynamically allocated String). b) Write functions create(), read() and display(); to create the calendar, to read the data from the keyboard and to print weeks activity details report on screen..	1	1	1
2	Develop a Program in C for the following operations on Strings. a. Read a main String (STR), a Pattern String (PAT) and a Replace String b. Perform Pattern Matching Operation: Find and Replace all occurrences of PAT in STR with REP if PAT exists in STR. Report suitable messages in case PAT does not exist in STR Support the program with functions for each of the above operations. Don't use Built-in functions.	1	1	1
3	Develop a menu driven Program in C for the following operations on STACK of Integers (Array Implementation of Stack with maximum size MAX) a. Push an Element on to Stack b. Pop an Element from Stack c. Demonstrate how Stack can be used to check Palindrome d. Demonstrate Overflow and Underflow situations on Stack e. Display the status of Stack f. Exit Support the program with appropriate functions for each of the above operations.	2	1	1
4	Develop a Program in C for converting an Infix Expression to Postfix Expression. Program should support for both parenthesized and free parenthesized expressions with the operators: +, -, *, /, % (Remainder), ^ (Power) and alphanumeric operands.	2	1	1
5	Develop a Program in C for the following Stack Applications a. Evaluation of Suffix expression with single digit operands and operators: +, -, *, /, %,^ b. Solving Tower of Hanoi problem with n disks	2	2	1
6	Develop a menu driven Program in C for the following operations on Circular QUEUE of Characters (Array Implementation of Queue with maximum size MAX) a. Insert an Element on to Circular QUEUE b. Delete an Element from Circular QUEUE c. Demonstrate Overflow and Underflow situations on Circular QUEUE d. Display the status of Circular QUEUE e. Exit Support the program with appropriate functions for each of the above operations.	2	2	1

7	Develop a menu driven Program in C for the following operations on Singly Linked List (SLL) of Student Data with the fields: USN, Name, Programme, Sem,PhNo a. Create a SLL of N Students Data by using front insertion. b. Display the status of SLL and count the number of nodes in it c. Perform Insertion / Deletion at End of SLL d. Perform Insertion / Deletion at Front of SLL(Demonstration of stack) e. Exit	2	2	1
8	Develop a menu driven Program in C for the following operations on Doubly Linked List (DLL) of Employee Data with the fields: SSN, Name, Dept, Designation,Sal, PhNo a. Create a DLL of N Employees Data by using end insertion. b. Display the status of DLL and count the number of nodes in it c. Perform Insertion and Deletion at End of DLL d. Perform Insertion and Deletion at Front of DLL e. Demonstrate how this DLL can be used as Double Ended Queue. f. Exit	2	2	1
9	Develop a Program in C for the following operationson Singly Circular Linked List (SCLL) with header nodesa. Represent and Evaluate a Polynomial P(x,y,z) = 6x2y2z-4yz5+3x3yz+2xy5z-2xyz3 b. Find the sum of two polynomials POLY1(x,y,z) and POLY2(x,y,z) and store the result in POLYSUM(x,y,z) Support the program with appropriate functions for each of the above operations.	2	3	1
10	Develop a menu driven Program in C for the following operations on Binary Search Tree (BST) of Integers . a. Create a BST of N Integers: 6, 9, 5, 2, 8, 15, 24, 14, 7, 8, 5, 2 b. Traverse the BST in Inorder, Preorder and Post Order c. Search the BST for a given element (KEY) and report the appropriate message d. Exit	3	3	1
11	Develop a Program in C for the following operations on Graph(G) of Cities a. Create a Graph of N cities using Adjacency Matrix. b. Print all the nodes reachable from a given starting node in a digraph using DFS/BFS method.	4	3	1
12	Given a File of N employee records with a set K of Keys (4-digit) which uniquely determine the records in file F. Assume that file F is maintained in memory by a Hash Table (HT) of memory locations with L as the set of memory addresses (2-digit) of locations in HT. Let the keys in K and addresses in L are Integers. Develop a Program in C that uses Hash function H: K →L as H(K)=K mod m (remainder method), and implement hashing technique to map a given key K to the address space L. Resolve the collision (if any) using linear probing.	4	3	1

EXPERIMENT WISE LESSON PLAN

Experiment No.1	
Name	Develop a Program in C for the following: a) Declare a calendar as an array of 7 elements (A dynamically Created array) to represent 7 days of a week. Each Element of the array is a structure having three fields. The first field is the name of the Day (A dynamically allocated String), The second field is the date of the Day (A integer), the third field is the description of the activity for a particular day (A dynamically allocated String). b) Write functions create(), read() and display(); to create the calendar, to read the data from the keyboard and to print weeks activity details report on screen..
Objectives	Introduce the data types structure dynamically allocation memory.
Experiment No.2	
Name	Develop a Program in C for the following operations on Strings. a. Read a main String (STR), a Pattern String (PAT) and a Replace String b. Perform Pattern Matching Operation: Find and Replace all occurrences of PAT in STR with REP if PAT exists in STR. Report suitable messages in case PAT does not exist in STR Support the program with functions for each of the above operations. Don't use Built-in functions.
Objectives	Understanding the implementation of string function‘s using arrays& pattern replacement methodology.
Experiment No.3	
Name	Develop a menu driven Program in C for the following operations on STACK of Integers (Array Implementation of Stack with maximum size MAX) a. Push an Element on to Stack b. Pop an Element from Stack c. Demonstrate how Stack can be used to check Palindrome d. Demonstrate Overflow and Underflow situations on Stack e. Display the status of Stack f. Exit Support the program with appropriate functions for each of the above operations.
Objectives	Understand the stack data structures & palindrome different functions onstacks i.e., push, pop display.
Experiment No.4	
Name	Develop a Program in C for converting an Infix Expression to Postfix Expression. Program should support for both parenthesized and free parenthesized expressions with the operators: +, -, *, /, % (Remainder), ^ (Power) and alphanumeric operands.
Objectives	Understand different notations to represent regular expression, infix to postfix conversion.
Experiment No. 5	
Name	Develop a Program in C for the following Stack Applications a. Evaluation of Suffix expression with single digit operands and operators: +, -, *, /, %,^

	b. Solving Tower of Hanoi problem with n disks.
Objectives	Understand different polish notation and evaluating postfix expression.
Experiment No.6	
Name	Develop a menu driven Program in C for the following operations on Circular QUEUE of Characters (Array Implementation of Queue with maximum size MAX) a. Insert an Element on to Circular QUEUE b. Delete an Element from Circular QUEUE c. Demonstrate Overflow and Underflow situations on Circular QUEUE d. Display the status of Circular QUEUE e. Exit Support the program with appropriate functions for each of the above operations.
Objectives	Understand the working of circularqueue
Experiment No.7	
Name	Develop a menu driven Program in C for the following operations on Singly Linked List (SLL) of Student Data with the fields: USN, Name, Programme, Sem,PhNo a. Create a SLL of N Students Data by using front insertion. b. Display the status of SLL and count the number of nodes in it c. Perform Insertion / Deletion at End of SLL d. Perform Insertion / Deletion at Front of SLL(Demonstration of stack) e. Exit
Objectives	Understand the Singly Linked List (SLL) data structures
Experiment No.8	
Name	Develop a menu driven Program in C for the following operations on Doubly Linked List (DLL) of Employee Data with the fields: SSN, Name, Dept, Designation,Sal, PhNo a. Create a DLL of N Employees Data by using end insertion. b. Display the status of DLL and count the number of nodes in it c. Perform Insertion and Deletion at End of DLL d. Perform Insertion and Deletion at Front of DLL e. Demonstrate how this DLL can be used as Double Ended Queue. f. Exit
Objectives	Understand the Doubly Linked List (DLL) data structures.
Experiment No.9	
Name	Develop a Program in C for the following operationson Singly Circular Linked List (SCLL) with header nodes a. Represent and Evaluate a Polynomial P(x,y,z) = 6x2y2z-4yz5+3x3yz+2xy5z-2xyz3 b. Find the sum of two polynomials POLY1(x,y,z) and POLY2(x,y,z) and store the result in

	POLYSUM(x,y,z) Support the program with appropriate functions for each of the above operations.
Objectives	Understand the working of Singly Circular Linked List,add two polynomialusingSCLL
Experiment No.10	
Name	Develop a menu driven Program in C for the following operations on Binary Search Tree (BST) of Integers. a. Create a BST of N Integers: 6, 9, 5, 2, 8, 15, 24, 14, 7, 8, 5, 2 b. Traverse the BST in Inorder, Preorder and Post Order c. Search the BST for a given element (KEY) and report the appropriate message d. Exit
Objectives	Understand the concept of Binary Search Tree (BST), different traversal methods.
Experiment No.11	
Name	Develop a Program in C for the following operations on Graph(G) of Cities a. Create a Graph of N cities using Adjacency Matrix. b. Print all the nodes reachable from a given starting node in a digraph using DFS/BFS method.
Objectives	Understand the concept of trees and adjacency matrix, Breath First Search(BFS) and Depth First Search.
Experiment No.12	
Name	Given a File of N employee records with a set K of Keys (4-digit) which uniquely determine the records in file F. Assume that file F is maintained in memory by a Hash Table (HT) of memory locations with L as the set of memory addresses (2-digit) of locations in HT. Let the keys in K and addresses in L are Integers. Develop a Program in C that uses Hash function H: K →L as H(K)=K mod m (remainder method), and implement hashing technique to map a given key K to the address space L. Resolve the collision (if any) using linear probing.
Objectives	Understand what is hashing and hashing function & concept of linear probing.

Introduction to Data Structure

Basic Concepts

The logical or mathematical model of a particular organization of data is called data structures. Data structures is the study of logical relationship existing between individual data elements, the way the data is organized in the memory and the efficient way of storing, accessing and manipulating the data elements.

Data Structures can be classifiedas:

- Primitive data structures
- Non-Primitive data structures.

Primitive data structures are the basic data structures that can be directly manipulated/operated by machine instructions. Some of these are character, integer, real, pointers etc.

Non-primitive data structures are derived from primitive data structures, they cannot be directly manipulated/operated by machine instructions, and these are group of homogeneous or heterogeneous data items. Some of these are Arrays, stacks, queues, trees, graphs etc.

Data structures are also classified as

•Linear data structures

•Non-Linear data structures.

In the Linear data structures processing of data items is possible in linear fashion, i.e., data can be processed one by one sequentially. Example of such data structures are:

•Array

•Linked list

•Stacks

•Queues

A data structure in which insertion and deletion is not possible in a linear fashion is called as nonlinear data structure. i.e., which does not show the relationship of logical adjacency between the elements is called as non-linear data structure. Such as trees, graphs and files.

Data structure operations:

The particular data structures that one chooses for a given situation depends largely on the frequency with which specific operations are performed.

Thefollowingoperationsplaymajorrolein theprocessingof data.

i) Traversing.

ii) Searching.

iii) Inserting.

iv) Deleting.

v) Sorting.

vi) Merging

STACKS:

A stack is an ordered collection of items into which new items may be inserted and from which items may be deleted at the same end, called the TOP of the stack. A stack is a non-primitive linear data structure.1 2 3 4 5

As all the insertion and deletion are done from the same end, the first element inserted into the stack is the last element deleted from the stack and the last element inserted into the stack is the first element to be deleted. Therefore, the stack is called Last-In First-Out (LIFO) data structure.

QUEUES:

A queue is a non-primitive linear data structure. Where the operation on the queue is based on First-In-First-Out FIFO process — the first element in the queue will be the first one out. This is equivalent to the requirement that whenever an element is added, all elements that were added before have to be removed before the new element can be removed.

For inserting elements into the queue are done from the rear end and deletion is done from the front end, we use external pointers called as rear and front to keep track of the status of the queue. During insertion, Queue Overflow condition has to be checked. Likewise during deletion, Queue Underflow condition is checked.

APPLICATION OF QUEUE

Queue, as the name suggests is used whenever we need to have any group of objects in an order in which the first one coming in, also gets out first while the others wait for their turn, like in the following scenarios:

- Serving requests on a single shared resource, like a printer, CPU task scheduling etc.
- In real life, Call Center phone systems will use Queues, to hold people calling them in an order, until a service representative is free.
- Handling of interrupts in real-time systems. The interrupts are handled in the same order as they arrive, First come first served.

LINKED LIST

Disadvantages of static/sequential allocation technique:

•If an item has to be deleted then all the following items will have to be moved by one allocation. Wastage of time.

•Inefficient memory utilization.

•If no consecutive memory (free) is available, execution is not possible.

Linear Linked Lists

Types of Linked lists:

•Single Linked lists

•Circular Single Linked Lists

•Double Linked Lists

•Circular Double Linked Lists.

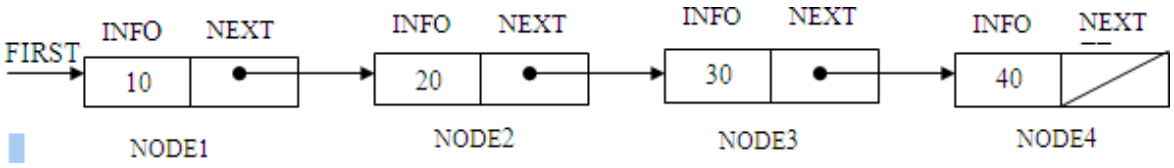
NODE:

Each node consists of two fields. Information (info) field and next address (next) field. The info field consists of actual information/data/item that has to be stored in a list. The second field next/link contains the address of the next node. Since next field contains the address,

It is of type pointer. Here the nodes in the list are logically adjacent to each other. Nodes that are physically adjacent need not be logically adjacent in the list.

The entire linked list is accessed from an external pointer FIRST that points to (contains the address of) the first node in the list. (By an ―external‖ pointer, we mean, one that is not included within a node. Rather its value can be accessed directly by referencing a variable).

The list containing 4 items/data 10, 20, 30 and 40 is shown below.



The nodes in the list can be accessed using a pointer variable. In the above fig. FIRST is the pointer having the address of the first node of the list, initially before creating the list, as list is empty? The FIRST will always be initialized to NULL in the beginning. Once the list is created, FIRST contains the address of the first node of the list.

The basic operations of linked lists are Insertion, Deletion and Display. A list is a dynamic data structure. The number of nodes on a list may vary dramatically as elements are inserted and deleted (removed).

The dynamic nature of list may be contrasted with the static nature of an array, whose size remains constant. When an item has to inserted, we will have to create a node, which has to be got from the available free memory of the computer system, so we shall use a mechanism to find an unused node which makes it available to us. For this purpose we shall use the get node operation (get node() function).

The C language provides the built-in functions like malloc(), calloc(), realloc() and free(), which are stored in stdlib.h header files. To dynamically allocate and release the memory locations from/to the computer system

TREES:

A data structure which is accessed beginning at the root node. Each node is either a leaf or an internal node. An internal node has one or more child nodes and is called the parent of its child nodes. All children of the same node are siblings. Contrary to a physical tree, the root is usually depicted at the top of the structure, and the leaves are depicted at the bottom. A tree can also be defined as a connected, acyclic di-graph.

Tree is a non-linear data structure which organizes data in hierarchical structure and this is a recursive definition.

A tree data structure can also be defined as follows...

Tree data structure is a collection of data (Node) which is organized in hierarchical structure and this is a recursive definition.

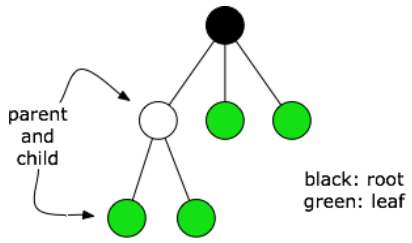


Figure: tree data structure

Graph:

Definition: A Graph G consists of two sets, V and E. V is a finite set of vertices. E is a set of edges or pair of vertices. Two types of graphs, Undirected Graph Directed Graph

Undirected Graph: In undirected graph the pair of vertices representing any edge is unordered. Thus the pairs (u,v)and (v,u) represent the same edge.

Directed graph: In a directed graph each edge is represented by a directed pair <u,v>, u is the tail and v is the head of the edge. Therefore <v,u>and <u,v> represent two different edges.

Graph representation can be done using adjacency matrix.

Adjacency matrix: Adjacency matrix is a two dimensional n x n array a, with the property that a[i][j]=1 iff the edge (i,j) is in E(G). a[i][j]=0 if there is no such edge in G.

Connectivity of the graph: A graph is said to be connected iff for every pair of distinct vertices u and v , there is a path from u to v.

Graph traversal can be done in two ways: depth first search(dfs) and breadth first search (bfs).

Hashing: Hashing is a process of generating key or keys from a string of text using a mathematical function called hash function. Hashing is a key-to-address mapping process.

Hash Table: In hashing the dictionary pairs are stored in a table called hash table. Hash tables are partitioned into buckets, buckets consists of s slots , each slot is capable of holding one dictionary pair.

Hash function: A hash function maps a key into a bucket in the hash table. Most commonly used hash function is,
h (k) = k % Dwhere,k is the keyD is Max size of hash table.

Collision Resolution: When we hash a new key to an address, collision may be created. There are several methods to handle collision, open addressing, linked lists and buckets.
In open addressing, several ways are listed, linear probing, quadratic probe, pseudorandom, and key offset.

Linear Probing: In linear probing, when data cannot be stored at the home address, collision is resolved by adding 1 to the current address.

PROGRAM:1

Develop a Program in C for the following:

a) Declare a calendar as an array of 7 elements (A dynamically Created array) to represent 7 days of a week. Each Element of the array is a structure having three fields. The first field is the name of the Day (A dynamically allocated String), the second field is the date of the Day (A integer), the third field is the description of the activity for a particular day (A dynamically allocated String).

b) Write functions create (), read() and display(); to create the calendar, to read the data from the keyboard and to print weeks activity details report on screen.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Day {
   char *name;
   int date;
   char *activity;
};

void create(struct Day *calendar) {
   for (int i = 0; i < 7; i++) {
      (calendar + i)->name = (char *)malloc(20 * sizeof(char));
      (calendar + i)->activity = (char *)malloc(100 * sizeof(char));
   }
}

void read(struct Day *calendar) {
   for (int i = 0; i < 7; i++) {
      printf("Enter name of day %d: ", i + 1);
      scanf("%s", (calendar + i)->name);
      printf("Enter date of day %d: ", i + 1);
      scanf("%d", &(calendar + i)->date);
      printf("Enter activity for day %d: ", i + 1);
      scanf(" %[^\n]s", (calendar + i)->activity);
   }
}

void display(struct Day *calendar) {
   printf("\nWeekly Activity Details:\n");
   for (int i = 0; i < 7; i++) {
      printf("Day %d: %s\n", i + 1, (calendar + i)->name);
```

```
        printf("Date: %d\n", (calendar + i)->date);
        printf("Activity: %s\n\n", (calendar + i)->activity);
    }
}

int main() {
    struct Day *calendar = (struct Day *)malloc(7 * sizeof(struct Day));
    create(calendar);
    read(calendar);
    display(calendar);

    // Free dynamically allocated memory
    for (int i = 0; i < 7; i++) {
        free((calendar + i)->name);
        free((calendar + i)->activity);
    }
    free(calendar);

    return 0;
}
```

Output 1

```
Enter name of day 1: Monday
Enter date of day 1: 1
Enter activity for day 1: pick and speak
Enter name of day 2: Tuesday
Enter date of day 2: 2
Enter activity for day 2: Reading
Enter name of day 3: Wednesday
Enter date of day 3: 3
Enter activity for day 3: writing
Enter name of day 4: Thursday
Enter date of day 4: 4
Enter activity for day 4: gaming
Enter name of day 5: Friday
Enter date of day 5: 5
Enter activity for day 5: playing
Enter name of day 6: Saturday
Enter date of day 6: 6
Enter activity for day 6: sleeping
Enter name of day 7: Sunday
Enter date of day 7: 7
Enter activity for day 7: watching movie
```

Weekly Activity Details:
Day 1: Monday
Date: 1
Activity: pick and speak

Day 2: Tuesday
Date: 2
Activity: Reading

Day 3: Wednesday
Date: 3
Activity: writing

Day 4: Thursday
Date: 4
Activity: gaming

Day 5: Friday
Date: 5
Activity: playing

Day 6: Saturday
Date: 6
Activity: sleeping

Day 7: Sunday
Date: 7
Activity: watching movie

PROGRAM: 2

Design, develop and implement a Program in C for the following operationsonStrings

a. **Read amainString(STR),aPatternString(PAT)andaReplaceString(REP)**

b. **Perform Pattern Matching Operation: Find and Replace all occurrencesofPATinSTRwithREPifPATexistsinSTR.Reportsuitablemessagesincase PAT doesnotexistinSTR**

Support the program with functions for each of the above operations. Don'tuseBuilt-in functions

```
#include<stdio.h>
void read();
void match();
char STR[100],PAT[100],REP[100],ANS[100];
int c,i,j,k,m,flag=0;
void main()
{
read();
match();
}
void read()
{
printf("enter the main string STR:"); gets(STR);
printf("enter pattern string PAT:"); gets(PAT);
printf("enter replace string REP:"); gets(REP);
}
void match()
{
c=i=j=k=m=0;
while(STR[c]!='\0')
{
 if(STR[m]==PAT[i])
{
i++;m++;
flag=1; if(PAT[i]=='\0')
{
for(k=0;REP[k]!='\0';k++,j++)
```

```
ANS[j]=REP[k]; i=0;
c=m;
}
}
else
{
ANS[j]=STR[c];
j++;c++;
m=c; i=0;
}
}
if(flag==0)
printf("pattern not found");
else
{
ANS[j]='\0';
printf("resultant string is %s",ANS);
}
}
```

Output 1

```
Enter the MAIN string:
KLS VDIT
Enter a PATTERN string:
KLS
Enter a REPLACE string:
CSE
The RESULTANT string is: CSE VDIT
```

Output 2

```
Enter the MAIN string:
Hi  cse
Enter a PATTERN string: hello
Enter a REPLACE string: kls
Pattern doesn't found!!!
```

PROGRAM:3

Design, Develop and Implement a menu driven Program in C for the following operations on STACK of Integers (Array Implementation of Stack with maximum size MAX)

a.Push an Element ontoStack

b. Pop an Element fromStack

c. Demonstrate how Stack can be used to check Palindrome

d. Demonstrate Overflow and Underflow situations on Stack

e. Display the status of Stack

f. Exit

Support the program with appropriate functions for each of the above operations.

```
#include<stdio.h>
#include<stdlib.h>
#define MAX 4
int stack[MAX],top=-1,item;
void push();
void pop();
void palindrome();
void display();
void main()
{
int choice;
while(1)
{
Printf(—------- STACK OPERATIONS  \n—);
printf("1.push\n 2.pop\n 3.palindrome\n 4.display\n 5.exit\n");
printf("enter choice");
scanf("%d",&choice);
switch(choice)
{
case 1:push(); break;
case 2:pop();
break;
case 3:palindrome(); break;
case 4:display(); break;
case 5:exit(0); break;
default:printf("invalid choice\n"); break;
}
}
```

```
}

void push()
{
if(top==MAX-1)
printf("stack overflow");
else
{
printf("enter the item to be pushed\n");
scanf("%d",&item);
top=top+1;
stack[top]=item;
}
}
void pop()
{
if(top==-1)
printf("stack underflow");
else
{
item=stack[top];
top=top-1;
printf("deleted item is %d",item);
}
}
void display()
{
int i; if(top==-1)
printf("stack is empty");
else
{
for(i=top;i>=0;i--)
printf("%d\t",stack[i]);
}
}
void palindrome()
{
int num[10],i=0,k,flag=1;
k=top;
```

```
while(k!=-1)
num[i++]=stack[k--];
for(i=0;i<=top;i++)
{
if(num[i]==stack[i])
continue;
else
flag=0;
}
if(top==-1)
 printf("stack is empty");
else
{
if(flag)
printf("palindrome");
else
printf("not a palindrome");
}
}
```

Output

```
------- STACK OPERATIONS ------
1.	Push
2.	Pop
3.	Palindrome
4.	Display
5.	Exit
Enter your choice 1
Enter element to be inserted 10
------- STACK OPERATIONS------
1.	Push
2.	Pop
3.	Palindrome
4.	Display
5.	Exit
Enter your choice 1
enter element to be inserted 20
------- STACK OPERATIONS------
```

```
1. Push
2. Pop
3. Palindrome
4. Display
5. Exit

Enter your choice 1
enter element to be inserted 30
------- STACK OPERATIONS------
1. Push
2. Pop
3. Palindrome
4. Display
5. Exit
Enter your choice 1
enter element to be inserted 40
------- STACK OPERATIONS------
1. Push
2. Pop
3. Palindrome
4. Display
5. Exit
Enter your choice 1
enter element to be inserted 50
------- STACK OPERATIONS------
1. Push
2. Pop
3. Palindrome
4. Display
5. Exit
Enter your choice 1 Stack Overflow:
------- STACK OPERATIONS------
1. Push
2. Pop
3. Palindrome
4. Display
5. Exit
Enter your choice 4
```

stack elements are:
50 40 30 20 10
------- STACK OPERATIONS------
1.Push 2.Pop
3. Palindrome
4. Display
5. Exit
Enter your choice 2 The poped element: 50
------- STACK OPERATIONS------
1.Push 2.Pop
3. Palindrome
4. Display
5. Exit
Enter your choice 2 The poped element: 40
------- STACK OPERATIONS------
1.Push 2.Pop
3. Palindrome
4. Display
5. Exit
Enter the choice 2
The poped element: 30
------- STACK OPERATIONS------
1. Push
2. Pop
3. Palindrome
4. Display
5. Exit
Enter your choice 2 The poped element: 20

PROGRAM 4

Design, develop and implement a Program in C for converting an Infix Expression to Postfix Expression. Program should support for both parenthesized and free parenthesized expressions with the operators: +, -, *,/, %(Remainder), ^ (Power) and alphanumeric operands.

```
#include<stdio.h>
#include<ctype.h>
#define SIZE 50
char s[SIZE];
int top=-1;
void push(char elem)
{
s[++top]=elem;
}
char pop()
{
return s[top--];
}
int pr(char elem)
{
switch(elem)
{
case '#':return 0;
case '(':return 1; case '+':
case '-':return 2; case '*':
case '/':
case '%':return 3;
case '^':return 4;
}
}
void main()
{
char infix[50],postfix[50],ch,elem;
int i=0,k=0;
printf("enter the infix expression\n");
gets(infix);
push('#');
```

```
 while((ch=infix[i++])!='\0')
 {
if(ch=='(')
push(ch);
else
if(isalnum(ch))
postfix[k++]=ch;
 else if(ch==')')
{
while(s[top]!='(')
postfix[k++]=pop();
elem=pop();
}
else
{
while(pr(s[top])>=pr(ch))
postfix[k++]=pop();
push(ch);
}
}
while(s[top]!='#')
postfix[k++]=pop();
postfix[k]='\0';
printf("infix expression is %s\n postfix expression is %s\n",infix,postfix);
}
```

Output1

enter the Infix Expression ((a+b)*c)
Given Infix Expn is: ((a+b)*c) The Postfix Expn is: ab+c*

Output 2

enter the Infix Expression (a+ (b-c)*d)
Given Infix Expn is: (a+ (b-c)*d) The Postfix Expn is: abc-d*+

PROGRAM 5:

Design, develop and implement a Program inC for the following Stack Applications
a.Evaluation of Suffixexpression withsingle digit operands and operators: +, -, *, /,%, ^
b.Solving Tower of Hanoi problem with n disks

```
#include<stdio.h>
#include<math.h>
#include<string.h>
int s[30],op1,op2;
int top=-1,i;
char p[30],sym;
int op(int op1,char sym,int op2){
switch(sym){
case '+':return op1+op2;
case '-':return op1-op2;
case '*':return op1*op2;
case '/':return op1/op2;
case '%':return op1%op2;
 case '^':
case '$':return pow(op1,op2);}
return 0;
}
int main(){
printf("\nEnter the valid postfix exp:");
scanf("%s",p);
for(i=0;i<strlen(p);i++)
{
sym=p[i];
if(sym>='0' &&sym<='9')
s[++top]=sym-'0';
else
{
op2=s[top--];
op1=s[top--];
s[++top]=op(op1,sym,op2);
}
}
printf("\nThe result is %d",s[top]);
```

Output1

Enter the valid postfix exp: 23+
The result is 5

Output2
Enter the valid postfix exp 123-4*+
The result is -3.

5b.Solving Tower of Hanoi problem with n disks

```
#include<stdio.h>
Voidtower(intn,charfrompeg,chartopeg,charauxpeg);
intn;
voidmain()
{
        printf("Entertheno.ofdiscs:\n");scanf("%d
        ",&n);
        printf("thenumberofmovesintowerofhenoiproblem\n");tower(n,'A','C','B');
}
voidtower(intn,charfrompeg,chartopeg,charauxpeg)
{
        if(n==1)
         {
          printf("movedisk1from%Cto%C\n",frompeg,topeg);return;
         }
         tower(n-1,frompeg,auxpeg,topeg);
        printf("movedisk%dfrom%CtoC\n",n,frompeg,topeg);tower(n-
        1,auxpeg,topeg,frompeg);
}
```

Output

```
Entertheno.ofdiscs:
3
the number of moves intower ofhenoiproblemMove disc 1
fromAtoC
Move disc 2 from A to BMove
disc 1 from C to BMove disc 3
from A to CMove disc 1 from
B to AMove disc 2 from B to
CMove disc 1 fromAtoC
```

PROGRAM 6

Design, develop and implement a menu driven Program in C for the following operations on Circular QUEUE of Characters (Array Implementation of Queue with maximum size MAX)

a Insert an Element on to Circular QUEUE
b Delete an Element from Circular QUEUE
c Demonstrate Overflow and Underflow situations on Circular QUEUE
d Display the status of Circular QUEUE
e Exit
Support the program with appropriate functions for each of the above operations.

```
#include<stdio.h>
#include<stdlib.h>
 #define MAX 5
Char q[MAX],item;
int f=0,r=-1,count=0;
void insert();
void delete();
void display();
void main()
{
int ch;
while(1)
{
printf("1.insert 2.delete 3.display 4.exit \n");
printf("enter choice\n");
scanf("%d",&ch);
switch(ch)
{
case 1:getchar();insert(); break;
case 2:delete(); break;
case 3:display(); break;
case 4:exit(0);
default :printf("Invalid choice\n"); break;

}
}
}
```

```
void insert()
{
if(count==MAX) printf("queue overflow\n");
else
{
printf("enter the item to be inserted\n");
 scanf("%c",&item );
r=(r+1)%MAX;
q[r]=item; count++;
}
}
void delete()
{
if(count==0)
printf("queue underflow\n"); else
{
printf("deleted item is %c\n",q[f]);
f=(f+1)%MAX;
count--;
}
}
void display()
{
int j=f,i;
if(count==0) printf("queue is empty\n");
else
{
printf("contents of circular queue\n");
for(i=1;i<=count;i++)
{
printf("%c\t",q[j]);
j=(j+1)%MAX;
}
printf("total number of items=%d\n",count);
}
}
```

Output

```
1.insert 2.delete 3.display 4.exit enter choice1
enter the item to be inserted A
1.insert 2.delete 3.display 4.exit enter choice1
enter the item to be inserted B
1.insert 2.delete 3.display 4.exit enter choice1
enter the item to be inserted C
1.insert 2.delete 3.display 4.exit enter choice1
enter the item to be inserted D
1.insert 2.delete 3.display 4.exit enter choice1
enter the item to be inserted E
1.insert 2.delete 3.display 4.exit enter choice1
queue overflow

1.insert 2.delete 3.display 4.exit enter choiceF
queue overflow
1.insert 2.delete 3.display 4.exit enter choice3
contents of circular queue
A B    C        D        E
total number of items=5
1.insert 2.delete 3.display 4.exitenter choice 2
deleted item is A
1.insert 2.delete 3.display 4.exit enter choice2
deleted item is B
1.insert 2.delete 3.display 4.exit enter choice2
deleted item is C
1.insert 2.delete 3.display 4.exit enter choice2
deleted item is D
1.insert 2.delete 3.display 4.exit enter choice2
deleted item is E
1.insert 2.delete 3.display 4.exit enter choice2
queue underflow
1.insert 2.delete 3.display 4.exit enter choice4
```

PROGRAM 7

Design, Develop and Implement a menu driven Program in C for the following operations on Singly Linked List (SLL) of Student Data with the fields: USN, Name, Branch, Sem, PhNo

a.Create a SLL of N Students Data by using front insertion.
b.Display the status of SLL and count the number of nodes in it
c.Perform Insertion and Deletion at End of SLL
d.Perform Insertion and Deletion at Front of SLL
eExit

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
void create();
void insert_front();
void insert_rear();
void display();
void delete_front();
void delete_rear();
int count=0;
struct node
{
char usn[20],name[50],branch[10];
 int sem;
unsigned long long int phno;
struct node *link;
};
struct node *first=NULL,*last=NULL,*temp=NULL,*p;
void main()
{
int ch,n,i;
while(1)
{
printf("1.create SLL 2.insert at front 3.insert at rear 4.display 5.delete at front 6.delete at rear 7.exit\n");
printf("enter choice\n");
scanf("%d",&ch);
switch(ch)
```

```
{

case 1:printf("enter the no.of students\n");
scanf("%d",&n);
for(i=1;i<=n;i++)
 insert_front(); break;
case 2:insert_front(); break;
case 3:insert_rear();break;
case 4:display();break;
case 5:delete_front();break;
case 6:delete_rear();break;
case 7:exit(0);
default:printf("invalid choice\n");break;
}
}
}
void create()
{
char usn[20],name[50],branch[10]; int sem;
unsigned long long int phno;
temp=(struct node*)malloc(sizeof(struct node));
printf("enter usn,name,branch,sem,phno\n");
scanf("%s%s%s%d%llu",usn,name,branch,&sem,&phno);
strcpy(temp->usn,usn);
strcpy(temp->name,name);
strcpy(temp->branch,branch);
temp->sem=sem;
temp->phno=phno;
count++;
}
void insert_front()
{
if(first==NULL)
{
create();
temp->link=NULL;
first=temp;
last=temp;
}
```

```
else
{
create();
temp->link=first;
first=temp;
}
}
void insert_rear()
{
if(first==NULL)
{
create();
temp->link=NULL; first=temp; last=temp;
}
else
{
create();
 temp->link=NULL;
last->link=temp;
last=temp;
}
}
void display()
{
if(first==NULL)
{
printf("list is empty\n");
}
else
{
p=first;
printf("content of list is\n"); while(p!=NULL)
{
printf("%s\t%s\t%s\t%d\t%llu\n",p->usn,p->name,p->branch,p->sem,p->phno);
p=p->link;
}
printf("total no.of students %d\n",count);
}
}
```

```
void delete_front()
{
p=first; if(first==NULL)
{
printf("list is empty\n");

}
else if(p->link==NULL)
{
printf("deleted node is %s\t%s\t%s\t%d\t%llu\n",p->usn,p->name,p->branch,p->sem,p->phno);
free(p);
first=NULL; count--;
}
else
{
first=p->link;
printf("deleted node is %s\t%s\t%s\t%d\t%llu\n",p->usn,p->name,p->branch,p->sem,p->phno);
free(p);
count--;
}
}
void delete_rear()
{
p=first;
if(first==NULL)
{
printf("list is empty\n");
}
else if(p->link==NULL)
{
printf("deleted node is %s\t%s\t%s\t%d\t%llu\n",p->usn,p->name,p->branch,p->sem,p->phno);
free(p);
first=NULL; count--;
 }
else
{
while(p->link!=last) p=p->link;
printf("deleted node is %s\t%s\t%s\t%d\t%llu\n",last->usn,last->name,last->branch,last-
>sem,last->phno); free(last);
```

```
p->link=NULL; last=p;
count--;
}
}
```

Output

```
1.create SLL 2.insert at front 3.insert at rear 4.display 5.delete at front 6.delete at rear 7.exit
enter choice1
enter the no.of students 2
enter usn,name,branch,sem,phno
2vd16cs024deepak cs39481830624

enter usn,name,branch,sem,phno
2vd17cs405yogesh cs49449323284

1.create SLL 2.insert at front 3.insert at rear 4.display 5.delete at front 6.delete at rear 7.exit
enter choice4
content of list is
2vd17cs405   yogesh cs      4        9449323284
2vd16cs024   deepak cs      3        9481830624

total no.of students 2
1.create SLL 2.insert at front 3.insert at rear 4.display 5.delete at front 6.delete at rear 7.exit
enter choice2
enter usn,name,branch,sem,phno  2vd15cs011suresh cs69481830624

1.create SLL 2.insert at front 3.insert at rear 4.display 5.delete at front 6.delete at rear 7.exit
enter choice4
content of list is
2vd15cs011   suresh   cs      6        9481830624
2vd17cs405   yogesh cs      4        9449323284
2vd16cs024   deepak cs      3        9481830624
total no.of students 3


1.create SLL 2.insert at front 3.insert at rear 4.display 5.delete at front 6.delete at rear 7.exit
enter choice4
content of list is
```

```
2vd15cs011  suresh  cs    6     9481830624
2vd17cs405  yogesh cs     4     9449323284
2vd16cs024  deepak cs     3     9481830624
2vd15cs048  vinay  cs     8     9481830624
total no.of students 4

1.create SLL 2.insert at front 3.insert at rear 4.display 5.delete at front 6.delete at rear 7.exit
enter choice5
deleted node is  2vd15cs011        suresh  cs    6     9481830624

1.create SLL 2.insert at front 3.insert at rear 4.display 5.delete at front 6.delete at rear 7.exit
enter choice6
deleted node is  monesh vinay      cs      8     9481830624
1.create SLL 2.insert at front 3.insert at rear 4.display 5.delete at front 6.delete at rear 7.exit
enter choice7
```

PROGRAM 8

Design, Develop and Implement a menu driven Program in C for the following operations on Doubly Linked List (DLL) of Employee Data with the fields: SSN, Name, Dept, Designation,Sal, PhNo

a. Create a DLL of N Employees Data by using end insertion.
b. Display the status of DLL and count the number of nodes in it
c. Perform Insertion and Deletion at End of DLL
d. Perform Insertion and Deletion at Front of DLL
e. Demonstrate how this DLL can be used as Double Ended Queue
f. Exit

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
 void create();
void insert_front();
void insert_rear();
void display();
void delete_front();
void delete_rear();
int count=0;
struct node
{
int ssn;
char name[50],dept[20],desg[20];
float sal;
unsigned long long int phno;
struct node*llink,*rlink;
};
struct node *first=NULL,*last=NULL,*temp;
void main()
{
int ch,n,i;
while(1)
{
printf("1.create\n 2.insert_front\n 3.insert_rear\n 4.display\n 5.delete_front\n 6.delete_rear\n
7.exit\n");
printf("enter choice\n");
```

```
 scanf("%d",&ch);
switch(ch)
{
case 1:printf("enter the number of employee\n");
scanf("%d",&n);
for(i=0;i<n;i++)
insert_rear(); break;
case 2:insert_front();break;
case 3:insert_rear();break;
case 4:display();break;
case 5:delete_front();break;
 case 6:delete_rear();break;
case 7:exit(0);
default:printf("invalid choice\n");break;
}
}
}
void create()
{
int ssn;
char name[50],dept[20],desg[20]; float sal;
unsigned long long int phno;
temp=(struct node*)malloc(sizeof(struct node)); temp->llink=temp->rlink=NULL;
printf("enter ssn,name,dept,desg,salaryand phno\n");
scanf("%d%s%s%s%f%llu",&ssn,name,dept,desg,&sal,&phno);
temp->ssn=ssn;
strcpy(temp->name,name); strcpy(temp->dept,dept);
strcpy(temp->desg,desg); temp->sal=sal;
temp->phno=phno; count++;
}
void insert_front()
{
if(first==NULL)
{
create();
first=temp;
last=temp;
 }
else
```

```
{
create();
temp->rlink=first;
first->llink=temp;
first=temp;
}
}
void insert_rear()
{
if(first==NULL)
{
create();
first=temp;
last=temp;
else{
create();
last->rlink=temp;
temp->llink=last;
temp->rlink=NULL;
last=temp;
}
}
}

void display()
{
struct node *p;
if(first==NULL)
{
printf("list is empty\n");
return;
}
p=first;
printf("contents of list\n");
while(p!=NULL)
{
printf("%d\t%s\t%s\t%s\t%f\t%llu\n",p->ssn,p->name,p->dept,p->desg,p->sal,p->phno); p=p-
>rlink;
}
```

```
printf("total no. of employee %d\n",count);
}

void delete_front()
{
struct node *p; if(first==NULL)
{
printf("list is empty,cannot delete\n");
}
else if(first->rlink==NULL)
{
printf("deleted data is %d\t%s\t%s\t%s\t%f\t%llu\n",first->ssn,first->name,first->dept,first-
>desg,first->sal,first->phno);
 first=NULL;
free(first); count--;
}
else
{
p=first; first=p->rlink;
printf("deleted data is %d\t%s\t%s\t%s\t%f\t%llu\n",p->ssn,p->name,p->dept,p->desg,p->sal,p-
>phno);
free(p);
count--;
}
}
void delete_rear()
{
struct node*p;
if(first==NULL)
{
 printf("list is empty,cannot delete\n");
}
else if(first->rlink==NULL)
{
printf("deleted data is %d\t%s\t%s\t%s\t%f\t%llu\n",first->ssn,first->name,first-
>dept,first->desg,first->sal,first->phno);
first=NULL;
free(first); count--;
}
```

```
else
{
p=last;
last=p->llink;
printf("deleted data is%d\t%s\t%s\t%s\t%f\t%llu\n",p->ssn,p->name,p->dept,p->desg,p->sal,p->phno);
free(p);
last->rlink=NULL; count--;
}
}
```

Output

```
1.create 2.insert_front 3.insert_rear 4.display 5.delete_front 6.delete_rear 7.exit
enter choice 1
enter the number of employee 2
enter ssn,name,dept,desg,salary and phno 120
yogesh csprogramer 140009481830624

enter ssn,name,dept,desg,salary and phno 110vinay csprogramer 140009481830625

1.create 2.insert_front 3.insert_rear 4.display 5.delete_front 6.delete_rear 7.exit
enter choice 2
enter ssn,name,dept,desg,salary and phno 111suresh cs lecturer 400009482230624

1.create 2.insert_front 3.insert_rear 4.display 5.delete_front 6.delete_rear 7.exit
enter choice 3
enter ssn,name,dept,desg,salary and phno 222naveen cs lecturer 500009482230677
1.      create 2.insert_front 3.insert_rear 4.display 5.delete_front 6.delete_rear 7.exit
enter choice 4

contents of list
111     suresh cs       lecturer        40000.000000 9482230624
120     yogesh cs       programer       14000.000000 9481830624
110     vinay cs        programer       14000.000000 9481830625
222     naveen cs       lecturer        50000.000000 9482230677
total no. of employee 4
1.create 2.insert_front 3.insert_rear 4.display 5.delete_front 6.delete_rear 7.exit
enter choice 5
deleted data is 111
```

```
1.create 2.insert_front 3.insert_rear 4.display 5.delete_front 6.delete_rear 7.exit
enter choice 4
contents of list
120    yogesh cs     programer    14000.000000 9481830624
110    vinay cs      programer    14000.000000 9481830625
222    naveen cs     lecturer     50000.000000 9482230677

total no. of employee 3
1.create2.insert_front 3.insert_rear 4.display 5.delete_front 6.delete_rear    7.exit
enter choice   6

deleted data is 222   naveen cs     lecturer        50000.000000 9482230677
1.create  2.insert_front 3.insert_rear 4.display 5.delete_fron  6.delete_rear 7.exit

enter choice   4
contents of list
120    yogesh cs     programer    14000.000000        9481830624
110    vinay cs      programer    14000.000000        9481830625
total no. of employee 2

1.create 2.insert_front 3.insert_rear 4.display 5.delete_front 6.delete_rear 7.exit

enter choice 7
```

PROGRAM 9

Design, Develop and Implement a Program in C for the following operations on Singly Circular Linked List (SCLL) with header nodes
a Represent and Evaluate a Polynomial P(x,y,z) = 6x2y2z-4yz5+3x3yz+2xy5z- 2xyz3
b Find the sum of two polynomials POLY1(x,y,z) and POLY2(x,y,z) and store the result in POLYSUM(x,y,z)

```
#include<stdio.h>
#include<stdlib.h>
 #include<math.h>
 struct node
{
int co,ex,ey,ez; struct node *link;
};
typedef struct node NODE;
NODE *createnode(int,int,int,int);
NODE *attachnode(NODE*,NODE*);
NODE *readpoly();
void display(NODE*);
void evaluate(NODE*);
NODE *addpoly(NODE*,NODE*,NODE*);
NODE *createnode(int co,int ex,int ey,int ez)
{
NODE *temp;
temp=(NODE*)malloc(sizeof(NODE)); temp->co=co;
temp->ex=ex; temp->ey=ey; temp->ez=ez; temp->link=NULL; return temp;
}

NODE *attachnode(NODE *temp,NODE *head)
{
NODE *cur; cur=head->link;
while(cur->link!=head)
{
cur=cur->link;
}
cur->link=temp; temp->link=head; return head;
}
```

```
NODE *readpoly()
{
int i,n,co,ex,ey,ez;
NODE *head=(NODE*)malloc(sizeof(NODE)); NODE *temp;
head->link=head;
printf("enter the number of terms\n");
scanf("%d",&n); for(i=0;i<n;i++)
{
printf("term %d\n",i+1);
printf("enter the coefficient\n");
scanf("%d",&co);
printf("enter exponent values of x,yand z\n");
scanf("%d%d%d",&ex,&ey,&ez);
temp=createnode(co,ex,ey,ez);
head=attachnode(temp,head);
}
return head;
}
void display(NODE *poly)
{
NODE *cur; cur=poly->link;
while(cur!=poly)
{
printf("%dx^%dy^%dz^%d+",cur->co,cur->ex,cur->ey,cur->ez);
cur=cur->link;
}
printf("\n");
}
void evaluate(NODE *poly)
{
NODE *cur; int x,y,z,res=0; cur=poly->link;
printf("enter the values of x,y,z\n");
scanf("%d%d%d",&x,&y,&z); while(cur!=poly)
{
 res+=cur->co*pow(x,cur->ex)*pow(y,cur->ey)*pow(z,cur->ez); cur=cur->link;
}
printf("result=%d\n",res);
}
```

```
NODE *addpoly(NODE *p1,NODE *p2,NODE *poly)
{
int comp;
NODE *a,*b,*temp; a=p1->link;
b=p2->link;
while(a!=p1&&b!=p2)
{
if(a->ex==b->ex && a->ey==b->ey && a->ez==b->ez) comp=0;
else if(a->ex>b->ex) comp=1;
else if(a->ex==b->ex && a->ey==b->ey) comp=1;
else if(a->ex==b->ex && a->ey==b->ey && a->ez>b->ez) comp=1;
else comp=-1;
switch(comp)
{
case 0:temp=createnode(a->co+b->co, a->ex, a->ey, a->ez); poly=attachnode(temp,poly);
a=a->link;
 b=b->link; break;
case 1:temp=createnode(a->co, a->ex,a->ey,a->ez);
poly=attachnode(temp,poly); a=a->link;
break;
case-1:temp=createnode(b->co,b->ex,b->ey,b->ez); poly=attachnode(temp,poly);
b=b->link; break;
}
}
while(a!=p1)
{
temp=createnode(a->co,a->ex,a->ey,a->ez); poly=attachnode(temp,poly);
a=a->link;
}
while(b!=p2)
{       temp=createnode(b->co,b->ex,b->ey,b->ez);

poly=attachnode(temp,poly); b=b->link;
}
return poly;
}
Void main()
{
```

```
int ch;
 NODE *p1,*p2,*p3; p3=(NODE*)malloc(sizeof(NODE)); p3->link=p3;
while(1)
{
printf("1.represent and evaluate 2.add two polynomial 3.exit\n");
printf("enter choice\n");
scanf("%d",&ch);
switch(ch)
{
case 1:printf("enter a polynomial\n");
p1=readpoly();
display(p1);
evaluate(p1);break;
case 2:printf("enter polynomial 1\n");
 p1=readpoly();display(p1);
printf(" enter polynomial 2\n");
p2=readpoly();
display(p2);
p3=addpoly(p1,p2,p3);
printf("the resultant polynomial is\n"); display(p3);break;
case 3:exit(0);
}
}
}
```

Output

```
1.represent and evaluate 2.add two polynomial 3.exit
enter choice1

enter a polynomial
enter the number of terms 5
term 1
enter the coefficient 6
enter exponent values of x,yand z 221
term 2
enter the coefficient-4
enter exponent values of x,yand z 015
term 3
enter the coefficient 3
```

```
enter exponent values of x,yand z 311

term 4
enter the coefficient 2
enter exponent values of x,yand z 151
term 5
enter the coefficient 2
enter exponent values of x,yand z 113

6x^2y^2z^1+-4x^0y^1z^5+3x^3y^1z^1+2x^1y^5z^1+2x^1y^1z^3+

enter the values of x,y,z
1
1
1
result=9
```

PROGRAM 10

Design, Develop and Implement a menu driven Program in C for the following operations on Binary Search Tree (BST) of Integers

A Create a BST of N Integers: 6, 9, 5, 2, 8, 15, 24, 14, 7, 8, 5, 2

B Traverse the BST in In-order, Preorder and PostOrder

C Searchthe BST for a givenelement (KEY) and reportthe appropriate message

D Delete anelement (ELEM) fromBST

EExit

```
#include <stdio.h>
#include <stdlib.h>
 int flag=0;
typedef struct BST
{
int data;
struct BST *lchild, *rchild;
}node;
void insert(node *, node *);
void inorder(node *);
void preorder(node *);
void postorder(node *);
node *search(node *, int, node * *);
void main()
{
int choice; int ans =1;
int key;
node *new_node, *root, *tmp, *parent; node *get_node();
root = NULL;
printf("\nProgram For Binary Search Tree ");
do {
printf("\n1.Create");
printf("\n2.Search");
printf("\n3.Recursive Traversals");
printf("\n4.Exit");
printf("\nEnter your choice :");
scanf("%d", &choice);
switch (choice)
{
case 1:do{
```

```
new_node = get_node();
printf("\nEnter The Element ");
scanf("%d", &new_node->data);
if (root == NULL)
root = new_node; else
insert(root, new_node);
printf("\nWant To enter More Elements?(1/0)");
scanf("%d",&ans);
}while (ans); break;
case 2:printf("\nEnter Element to be searched :");
scanf("%d", &key);
tmp = search(root, key, &parent);
if(flag==1)
{
 printf("\nParent of node %d is %d", tmp->data, parent->data);
}
else
{
printf("\n The %d Element is not Present",key);
}
flag=0; break;
case 3:if (root == NULL)
printf("Tree Is Not Created");
else
{
printf("\nThe Inorder display :\n "); inorder(root);
printf("\nThe Preorder display : \n"); preorder(root);
printf("\nThe Postorder display : \n"); postorder(root);
}
break;
}
} while (choice != 4);
}
node *get_node()
{
node *temp; temp = (node *)malloc(sizeof(node));
temp->lchild = NULL; temp->rchild = NULL; return temp;
}
```

```
void insert(node *root, node *new_node)
{
if (new_node->data < root->data)
{
if (root->lchild == NULL) root->lchild = new_node; else
insert(root->lchild, new_node);
}
if (new_node->data > root->data)
{
if (root->rchild == NULL) root->rchild = new_node; else
insert(root->rchild, new_node);
}
}
node *search(node *root, int key, node **parent)
{
node *temp; temp = root;
while (temp != NULL)
{
if (temp->data == key)
 {
printf("\nThe %d Element is Present", temp->data);
flag=1;
return temp;
}
*parent = temp;
if (temp->data > key) temp = temp->lchild; else
temp = temp->rchild;
}
return NULL;
}
void inorder(node *temp)
{
if (temp != NULL)
{
inorder(temp->lchild); printf("%d\t", temp->data); inorder(temp->rchild);
}
}
```

```
void preorder(node *temp)
{
if (temp != NULL)
{
printf("%d\t",temp->data); preorder(temp->lchild); preorder(temp->rchild);
 }
}
void postorder(node *temp)
{
if (temp != NULL)
{
postorder(temp->lchild); postorder(temp->rchild); printf("%d\t", temp->data);
}
}
```

Output

Program For Binary Search Tree 1.Create2.Search3.Recursive Traversals 4.Exit
Enter your choice :1
Enter The Element 6
Want To enter More Elements?(1/0)1 Enter The Element 9
Want To enter More Elements?(1/0)1 Enter The Element 5
Want To enter More Elements?(1/0)1 Enter The Element 2
Want To enter More Elements?(1/0)1 Enter The Element 8
Want To enter More Elements?(1/0)1

Enter The Element 15
Want To enter More Elements?(1/0)1 Enter The Element 24
Want To enter More Elements?(1/0)1 Enter The Element 14
Want To enter More Elements?(1/0)1 Enter The Element 7
Want To enter More Elements?(1/0)1 Enter The Element 8
Want To enter More Elements?(1/0)1 Enter The Element 5
Want To enter More Elements?(1/0)1 Enter The Element 2
Want To enter More Elements?(1/0)0

1.Create2.Search3.Recursive Traversals 4.Exit
Enter your choice :3

The Inorder display :
2 5 6 7 8 9 14 15 24
The Preorder display :
6 5 2 9 8 7 15 14 24
The Postorder display :
2 5 7 8 14 24 15 9 6

1.Create2.Search3.Recursive Traversals 4.Exit
Enter your choice :2
Enter Element to be searched:15

The 15 Element is Present Parent of node 15 is 9

1.Create2.Search3.Recursive Traversals 4.Exit
Enter your choice :2
Enter Element to be searched :24

The 24 Element is Present Parent of node 24 is 15

1.Create2.Search3.Recursive Traversals 4.Exit
Enter your choice :2
Enter Element to be searched :2

1.Create2.Search3.Recursive Traversals 4.Exit
Enter your choice :4

PROGRAM 11

Design, develop and implement a Program in C for the following operations on Graph (G) of Cities

A Create a Graph of N cities using Adjacency Matrix.

BPrintall the nodes reachable from a givenstarting node ina digraph using BFS method

CCheck whether a given graph is connected or notusingDFS Method.

```
#include<stdio.h>
#include<stdlib.h>
int n,a[10][10],i,j,source,s[10],choice,count;
void bfs(int n,int a[10][10],int source,int s[])
{
int q[10],u;
int front=1,rear=1;
s[source]=1;
q[rear]=source;
 while(front<=rear)
{
u=q[front]; front=front+1;
for(i=1;i<=n;i++)
if(a[u][i]==1 &&s[i]==0)
{
rear=rear+1; q[rear]=i;
s[i]=1;
}
}
}
void dfs(int n,int a[10][10],int source,int s[])
{
s[source]=1; for(i=1;i<=n;i++)
if(a[source][i]==1 && s[i]==0) dfs(n,a,i,s);
}
int main()
{
printf("Enter the number of nodes : \n");
scanf("%d",&n);
printf("\n Enter the adjacency matrix\n");
for(i=1;i<=n;i++)
```

```
for(j=1;j<=n;j++)
scanf("%d",&a[i][j]);
while(1)
{
printf("\n\n1.BFS\n 2.DFS\n 3.Exit\n");
printf("\nenter your choice\n");
scanf("%d",&choice);
switch(choice)
{
case 1: printf("\n Enter the source :\n");
scanf("%d",&source);
for(i=1;i<=n;i++)
s[i]=0;
bfs(n,a,source,s);
for(i=1;i<=n;i++)
{
if(s[i]==0)
printf("\n The node %d is not reachable\n",i);
else
printf("\n The node %d is reachable\n",i);
}
break;
case 2:printf("\nEnter the source vertex :\n");
scanf("%d",&source);
count=0; for(i=1;i<=n;i++) s[i]=0;
dfs(n,a,source,s);
for(i=1;i<=n;i++)
if(s[i])
count=count+1;
if(count==n)
printf("\nThe graph is connected.");
else
printf("\nThe graph is not connected."); break;
case 3: exit(0);
 }
}
}
```

Output1

```
Enter the number of nodes: 4
Enter the adjacencymatrix 0 0 1 0
1 0 1 0
0 0 0 0
0 0 0 0

enter your choice 1

Enter the source :1
The node 1 is reachable
The node 2 is notreachable
The node 3 is reachable
The node 4 is notreachable

1.BFS2.DFS3.Exit
enter your choice 2
```

Output2

```
Enter the number of nodes : 3
Enter theadjacencymatrix 0 1 1
0 0 0
0 0 0
1.BFS2.DFS3.Exit
enter your choice 1
Enter the source : 1
The node 1 is reachable
The node 2 is reachable
The node 3 is reachable
```

PROGRAM 12

Given a File of N employee records with a set K of Keys(4-digit) which uniquely determine the records in file F. Assume that file F is maintained in memory by a Hash Table(HT) of memory locations with L as the set of memory addresses (2-digit) of locations in HT. Let the keys in K and addresses in L are Integers.

Design and develop a Program in C that uses Hash function H: K->L as H(K)=K mod m (remainder method), and implement hashing technique to map a given key K to the address space L. Resolve the collision (if any) using linear probing.

```
#include<stdio.h>
 #include<stdlib.h>
#define MAX 100
void display(int a[MAX]);
int create(int num);
void linearprob(int a [MAX],int key,int num);
void main()
{
int a[MAX],i,num,key,ans=1;
printf("collission hanmdling by linear probing\n");
for(i=0;i<MAX;i++)
a[i]= -1;
do{
printf("enter the data\n");
scanf("%4d",&num);
key=create(num);
linearprob(a,key,num);
printf("do yuou want to comntinue[1/0]\n");
scanf("%d",&ans);
}while(ans);
display(a);
}
 int create(int num)
{
int key; key=num%100; return key;
}
void linearprob(int a[MAX],int key, int num)
{
```

```
int flag=0,count=0,i;
if(a[key]==-1)
a[key]=num;
else
{
printf("\n collision deleted\n");
i=0;
while((i<key)&&(flag==0))
{
if(a[i]==-1)
{
a[i]=num; flag=1; break;
} i++;
}
 }
}
void display(int a[MAX])
{
int ch,i;
printf("\n 1.display all 2.filtered display\n");
printf("enter choice\n");
scanf("%d",&ch); if(ch==1)
{
for(i=0;i<MAX;i++)
printf("%d\t %d\n",i,a[i]);
}
else
{
for(i=0;i<MAX;i++)
{
if(a[i]!=-1)
{
printf("%d\t %d\n",i,a[i]); continue;
}
}
}
}
```

output

collision handling by linear probing : Enter the data
1234
Do you wish to continue ? (1/0) 1
Enter the data 2548
Do you wish to continue ? (1/0) 1
Enter the data 3256
Do you wish to continue ? (1/0) 1
Enter the data 1299
Do you wish to continue ? (1/0) 1
Enter the data 1298
Do you wish to continue ? (1/0) 1
Enter the data 1398
Collision Detected...!!!

Collision avoided successfully using LINEAR PROBING Do you wish to continue ? (1/0)
0

1.Display ALL 2.Filtered Display

enter the choice2
the hash table is
0 1398
34 1234
48 2548
56 3256
57 1456
98 1298
99 1299

Virtual Lab Experiments

EXPERIMENT:01

Linked List

Linked list is a linear data structure where each data is a separate object (of same data type). Linked list objects do not occupy the contiguous memory location as compared to the array which is also a linear data structure where elements have contiguous memory allocation; instead linked lists are linked using pointers. Elements of the linked lists are known as Nodes.

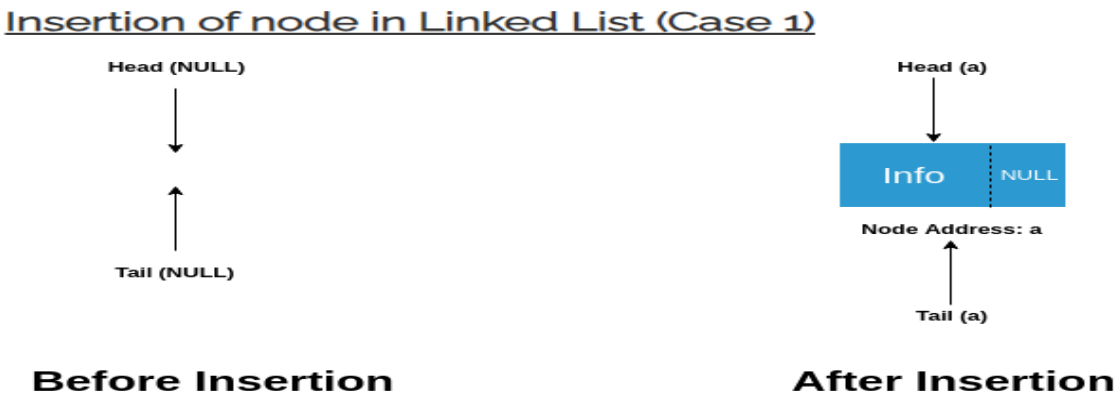
Steps of simulator:

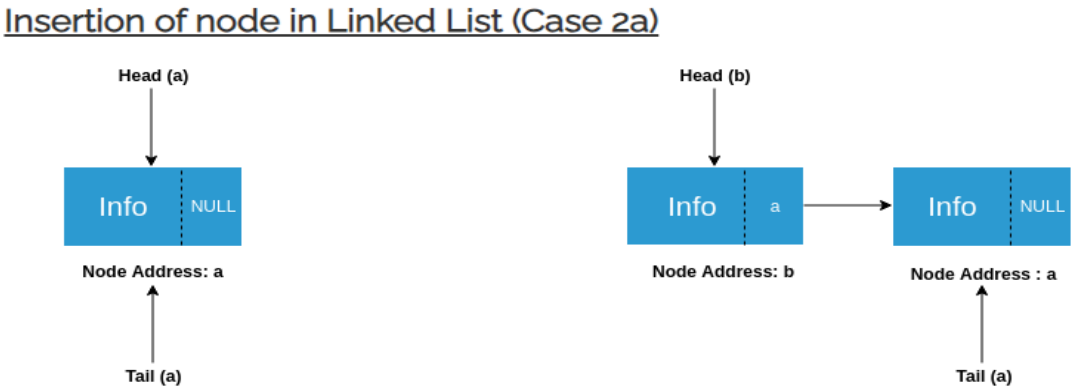
Insertion of elements into Singly Linked List
Two cases may arise when we want to insert data at the front.

Case – 1 : When the list is empty i.e. head = NULL

Here we can simply do head = tail = newNode(pointer)

Case – 2 : When list contains some entries i.e. head != NULL
Insertion at Front
Insertion at End
Insertion after a Node

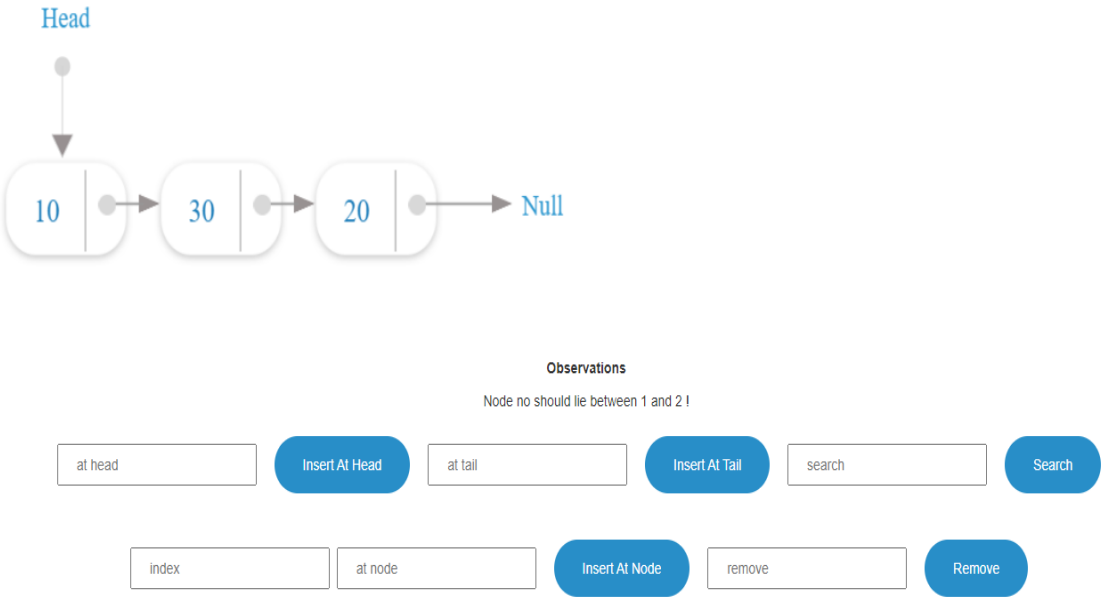




Here the changes we made are

- Set the next of **newNode** = **head**
- Set **head** = **newNode** (pointer)

Simulation Link:https://ds1-iiith.vlabs.ac.in/exp/linked-list/singly-linked-list/sllpractice.html



EXPERIMENT:02

Binary Search Tree

A binary search tree is a rooted binary tree, whose internal nodes each store a key (and optionally, an associated value) and each have two distinguished sub-trees, commonly denoted left and right. The tree additionally satisfies the binary search property, which states that the key in each node must be greater than or equal to any key stored in the left sub-tree, and less than or equal to any key stored in the right sub-tree. The leaves (final nodes) of the tree contain no key and have no structure to distinguish them from one another.

Steps of simulator:

Inserting a new node into a BST

- ✓ Start from root.
- ✓ Compare the inserting element with root and if it's lesser than root, then recurse for left, else recurse for right.
- ✓ After reaching the end, just insert that node at left(if less than current), else right.

Simulation Link:https://ds1-iiith.vlabs.ac.in/exp/binary-search-trees/bst-insert/bstInsert.html